

Mystes: Practical, Secure, and Fast System Call Interposition

Anonymous Author(s)

Abstract

System call interposition is a key technique for enforcing security policies, enabling compatibility layers, and supporting broader system functionality. However, existing approaches struggle to jointly achieve strong security and high performance. Moreover, approaches that attempt to provide both require intrusive changes to applications or platforms, such as OS or hardware modifications. We present *Mystes*¹, to the best of our knowledge, the first system call interposition approach to provide both without intrusive modifications. *Mystes* achieves this through a new interposition design that combines multiple interposition mechanisms with enhanced in-process isolation, built entirely on commodity Linux and hardware primitives. We implement *Mystes* as a Linux prototype and build three case studies on top of it: two that improve the effectiveness of existing defenses — system call filters and eXecute-Only Memory (XOM) — and a third demonstrating transparent integration of a high-performance user-space TCP/IP stack. We evaluate its performance using microbenchmarks and real-world workloads. Our results show that *Mystes* prevents all known and newly identified attacks while approximating the performance of non-security-focused interposers, demonstrating that strong security need not come at the cost of performance or require intrusive changes.

1 Introduction

Over the years, system call interposition has emerged as a versatile mechanism for observing and influencing application behavior across a wide range of settings. By mediating interactions between applications and the OS, interposition has enabled dynamic analysis [4, 9, 18, 22], facilitated compatibility with alternative OS abstractions [21, 25, 45, 65, 82, 93, 131], and enabled designs that shift functionality out of the OS kernel [8, 50, 99, 134]. At the same time, system call interposition has played a central role in security- and reliability-focused systems, including software updating [61, 62, 100], sandboxing [72, 106, 115, 119, 120, 130], intrusion detection [76, 87, 88, 92, 103, 121], as well as fuzzing and sanitization [68, 129], and energy- and carbon-aware monitoring and estimation techniques [27, 42, 54, 63, 97, 105, 112].

Despite their central role across these settings, Linux’s built-in interfaces for system call mediation impose fundamental trade-offs between performance, usability, and security. Specifically, `ptrace` [16], `seccomp` [19], and `Syscall User`

`Dispatch (SUD)` [23] suffer from well-known performance and expressivity limitations [59, 64, 106, 131]. To overcome these limitations, several prior systems rely on application or platform modifications, such as kernel or hardware changes, or custom `libOS` and `libc` environments [32, 53, 57, 98, 106, 107, 110, 115, 119, 130]. While such approaches can improve performance or strengthen security guarantees, they do so at the cost of increased development and deployment complexity, reduced portability, higher maintenance overhead, and an expanded trusted computing base (TCB). This tension is particularly acute in performance-critical software such as web servers and databases, where strict performance requirements must coexist with strong security guarantees and usability remains a key practical constraint.

To meet the usability and performance requirements of modern critical infrastructure, prior work has explored system call interposition entirely in user space through binary rewriting [24, 31, 59, 62, 64, 94, 100, 110, 131]. By rewriting *syscall instructions*² to redirect execution to user-space handlers, these approaches avoid additional mode and context switches while placing the interposition logic within the same address space as the application, enabling low-overhead mediation. However, such approaches face two fundamental problems. First, existing systems rely on brittle heuristics, such as those used to locate and patch *syscall instructions*, which can lead to code and data corruption [59]. Second, the interposition logic executes within the same address space as the application it mediates. Although this in-process placement is crucial for achieving low overhead and compatibility, it significantly enlarges the attack surface: a compromised application can directly access, tamper with, or corrupt the interposer’s own state.

In this paper, we present *Mystes*, a system call interposition approach designed to provide strong security and high performance. *Mystes* combines multiple Linux interposition interfaces for comprehensive mediation, binary rewriting for low-overhead interception, and PKU [2]-based isolation and sandboxing to isolate and protect the interposer from a compromised application. By relying exclusively on commodity mechanisms, *Mystes* avoids common barriers to practical deployment, such as custom toolchains, application modifications, and OS or hardware changes. We build three case studies on top of *Mystes*: adaptive system call filtering [51], enhanced eXecute-Only Memory (XOM) [135], and transparent integration with a user-space network stack [131].

¹From the Ancient Greek term *mystes*, meaning a person initiated into deeper or hidden knowledge.

²Throughout this paper, we use *syscall instructions* to refer to x86-64 instructions that trigger system calls (i.e., `syscall` and `sysenter`).

In summary, this paper makes the following contributions:

- We present *Mystes*, to the best of our knowledge, the first system call interposition approach that jointly provides strong security and high performance for unmodified Linux applications and platforms.
- We demonstrate the breadth of *Mystes* through three case studies: adaptive system call filtering, enhanced eXecute-Only Memory (XOM), and transparent integration with a user-space network stack.
- We evaluate *Mystes*' security guarantees against known and new attacks, and its performance on microbenchmarks and real-world system call-intensive workloads, demonstrating that it achieves low overhead with strong security guarantees.

We will release the source code upon publication.

2 Background

System call interposition is the ability to intercept system calls issued by an application and execute custom logic in place of, or alongside, the system call. Over the years, various designs have emerged to realize this capability, mainly differing in where and how interposition occurs.

2.1 Linux Interfaces

Traditionally, system call interposition has been realized through cross-process designs, where the monitor runs in a separate process from the monitored application. The canonical example is *ptrace* [16], which allows a *tracer* process to observe and control the execution of a *tracee* process. Because the tracer runs in a separate process, *ptrace* benefits from strong isolation, preventing the tracee from directly tampering with the interposition logic, and supports expressive mediation. Due to this flexibility, *ptrace* has been widely used in debugging [4, 9, 18], security [37, 68, 72, 115, 117–119], and reliability systems [61, 89]. However, *ptrace* incurs substantial overhead due to frequent context switches between tracer and tracee, making it unsuitable for performance-critical workloads [43, 59, 64, 106, 115, 131].

Another Linux interface, *seccomp* [19], provides mechanisms for restricting or mediating system calls issued by a process. In particular, *seccomp-bpf* allows processes to install in-kernel filters that are evaluated before a system call is executed. Because these filters run inside the kernel, *seccomp-bpf* introduces minimal runtime overhead and benefits from strong isolation from the application, preventing untrusted code from tampering with the filtering logic. As a result, *seccomp-bpf* is widely used as a security primitive in sandboxing and system call filtering mechanisms [44, 55, 56, 104]. However, *seccomp-bpf* also has inherent limitations. For example, it cannot inspect memory referenced by pointer arguments, preventing the enforcement of fine-grained security policies [59, 64, 106, 131]. Additionally, its hierarchical design forces processes to inherit the

system call filters of their ancestors, limiting the ability to dynamically adapt policies for newly executed programs [51].

To address the overhead of *ptrace* and the expressiveness limitations of *seccomp-bpf*, mechanisms such as *seccomp-user* and *SUD* [23] redirect system calls from kernel space to user space for mediation. In these designs, intercepted system calls trigger a user-space handler that can inspect arguments and implement more expressive policies. *SUD*, in particular, delivers a *SIGSYS* signal when a system call is issued, transferring control from the kernel to a user-space handler. Interposition can be dynamically controlled through a selector byte in user space and optionally bypassed for system calls issued from designated code regions, providing greater flexibility than *seccomp-user*. Due to this flexibility, *SUD* has been adopted in various contexts, including OS compatibility layers [21, 25]. However, such mechanisms require extra mode switches between kernel and user space, which can incur significant costs in system call-intensive workloads [59, 64, 131]. Moreover, because the interposition logic executes within the same address space as the application, attackers who gain control of the application can directly manipulate, interfere with, or bypass the interposer, undermining the mediation layer's security.

2.2 Binary Rewriting

Binary rewriting enables system call interposition by modifying binaries such that *syscall* instructions transfer control to the interposition logic before execution of the system call. Because the interposer runs entirely in user space, rewriting-based approaches avoid extra context and mode switches, enabling very low runtime overhead while preserving deployability without kernel modifications [24, 31, 62, 100, 101]. However, building reliable binary rewriting-based interposition mechanisms remains challenging in practice, particularly on architectures such as x86-64 with variable-length instructions, where accurate instruction disassembly and rewriting are inherently difficult [29, 31, 96].

To address these challenges, prior systems rely on various heuristics and assumptions about program structure and compiler behavior [31]. In particular, *syscall* instructions are only two bytes long, whereas typical redirection instructions are longer, often requiring relocation of surrounding instructions and introducing correctness issues when assumed code patterns do not hold. To overcome this, *zpoline* [131] avoids instruction relocation altogether by rewriting *syscall* instructions into a two-byte *callq *%rax* and installing a trampoline in the low virtual address space. Since the x86-64 calling convention stores the *syscall* number in *rax*, the resulting transfer jumps to an address mapped to the interposition logic through the trampoline, resolving the instruction size mismatch.

Despite its advantages, *zpoline* still depends on accurate static disassembly and cannot interpose system calls from

dynamically generated or loaded code. lazypoline [64] addresses this by using SUD to intercept syscall instructions at their first attempted execution and rewrite them on the fly, while SUD remains as a fallback mechanism for any undiscovered syscall instructions. Nevertheless, recent work [59] discovered multiple issues in both approaches [64, 131]. For example, on-the-fly binary rewriting is fundamentally fragile and can even lead to data and code corruption. Even though the authors provide solutions, these still require representative inputs to identify syscall instructions ahead of time, which may not always be feasible in practice. Regardless of the rewriting strategy, all binary rewriting-based interposers share the same fundamental in-process security limitation as SUD and seccomp-user (see Section 2.1).

2.3 In-Process Isolation

The security limitations of in-process designs call for isolation primitives that can protect the interposition logic from the application within the same address space. Various hardware and software mechanisms provide such guarantees [28, 30, 46, 71, 75, 78–81, 84, 90, 102, 113, 114, 124, 126–128]. In this work, we focus on Protection Keys for Userspace (PKU) [2], a hardware primitive available on commodity x86-64 machines, with support coming to ARM [30] and existing implementations for RISC-V [107]. PKU allows memory pages to be associated with protection keys, with access permissions controlled through the PKRU register via the wrpkru instruction. Because permission changes require no kernel transitions, PKU enables fast user-space domain switches. Applications assign protection keys to memory regions via pkey_mprotect [11], enabling hardware-backed isolation between components within the same address space.

Unfortunately, PKU alone is not sufficient to achieve strong isolation. Because the wrpkru instruction can be executed from user space without privilege, a compromised application can directly modify key permissions to bypass isolation. Additionally, the kernel can be exploited as a confused deputy: system calls that affect memory mappings or protection key assignments (e.g., mmap, pkey_mprotect) can be used to undermine PKU-based boundaries [43, 119]. To address these issues, an accompanying sandbox is required to vet potentially dangerous instructions and OS abstractions. However, existing implementations of such sandboxes either operate as cross-process monitors [115, 119], reintroducing the overhead that in-process designs seek to avoid, or rely on custom toolchains and libOS environments [130] or kernel modifications [106, 115], limiting their deployability.

3 Threat Model

We make no assumptions about the application code: it may contain exploitable vulnerabilities and execute arbitrary attacker-controlled code. Our trusted computing base (TCB) consists of the components of Mystes and the OS

kernel, both of which are assumed to be free of bugs and exploitable vulnerabilities. We further assume that PKU [2] is available on the target platform and correctly implemented. We do not rely on complementary defenses such as software diversity [77] or control-flow integrity (CFI) [26, 39], which may raise the bar for exploitation but are not assumed to be present or correctly enforced. Attacks targeting the underlying hardware, such as transient execution attacks [73, 83] and fault injection attacks [36, 70, 108, 116], are out of scope. Denial-of-service (DoS) attacks are also out of scope. Our threat model is consistent with prior work [60, 106, 115, 130].

4 Design Goals

We aim to build a system call interposition approach that overcomes the limitations of prior work (see Section 2), targeting the following design goals:

- **Unmodified Applications and Platforms.** The interposer should support unmodified applications and operate without requiring changes to the software or hardware stack, including the OS, libc, existing defenses, and tooling.
- **Expressive Interposition Logic.** The interposer must support fine-grained mediation policies.
- **Low Runtime Overhead.** The design should maintain low runtime overhead, even under system call-intensive workloads commonly found in critical infrastructure.
- **Strong Resource Isolation.** The interposer treats the application as fully untrusted and must not expose critical resources to it. In particular, the interposer’s internal state must be protected by default to prevent tampering or leakage.

Design Rationale. Achieving low overhead demands an in-process design, which in turn requires binary rewriting to redirect system calls without introducing additional context and mode switches. Yet, binary rewriting is extremely fragile and error-prone, even with recent advances in the field [59, 64, 131]. Placing the interposition logic in-process, however, exposes it to a compromised application, necessitating in-process isolation (see Section 2.3). In this work, we focus on PKU [2]. Finally, PKU requires an additional security layer against attackers who aim to undermine its isolation guarantees [43, 106, 119, 130]. These insights drive the design of Mystes.

5 Mystes

We present Mystes, a practical system call interposition approach that provides strong security guarantees while preserving high performance. Mystes combines multiple Linux interfaces with binary rewriting to ensure comprehensive system call mediation throughout the application lifetime. To enforce strong isolation, Mystes leverages PKU and sandboxing to protect its in-process interposition logic.

5.1 Interposition Design

Mystes consists of two interposition components: CP-MON, a cross-process interposer that provides strong isolation by design, and IP-MON, a fast in-process interposer. Until IP-MON is initialized, application system calls cannot yet be mediated in-process. Prior approaches close this gap using custom loaders, libOS environments, or kernel changes [60, 106, 115, 119, 130], but this increases deployment complexity, reduces portability, and expands the trusted computing base (see Section 4). To avoid these drawbacks, Mystes instead relies on CP-MON, an independent cross-process interposer that requires no cooperation from the target process and mediates its system calls until IP-MON takes over.

Figure 1 illustrates the control flow during application startup. To launch an application under Mystes, a Launcher process first spawns CP-MON as a child (step ①). CP-MON then attaches to the Launcher via `PTRACE_SEIZE` (step ②), enabling it to intercept system calls. The Launcher next calls `execve` to become the target application (step ③), at which point CP-MON begins mediating its system calls. Eventually, IP-MON is loaded and initialized (step ④). Once ready, IP-MON signals CP-MON to detach and takes over mediation for the remainder of execution (step ⑤). Although CP-MON incurs high overhead, it is active only during this early startup period, typically a small fraction of total execution time.

The Launcher and CP-MON are compiled as standalone executables. IP-MON is an in-process monitor that must be injected into the application’s address space. For dynamically linked applications, similar to prior work [59, 64, 131], IP-MON is injected via `LD_PRELOAD` [6], a standard mechanism to load a shared library into the process during dynamic loading. Binary rewriting is essential to achieve low-overhead interposition — by redirecting syscall instructions to the interposer in user space, it avoids additional context and mode switches. However, existing state-of-the-art approaches face several challenges (see Section 2.2). IP-MON uses *zpoline*-like binary rewriting [131] to efficiently redirect syscall instructions to the interposition logic, with SUD as a fallback to ensure comprehensive mediation with minimal performance impact (see Figure 2). Once control flow is redirected to the interposer, whether through binary rewriting or SUD, IP-MON disables SUD-based interposition via the selector, preventing the interposer’s own system calls from being intercepted. SUD-based interposition is re-enabled immediately before returning control to the application.

A key challenge in binary rewriting-based approaches is preventing the interposer’s own syscall instructions from being rewritten, which would result in recursive control flows and non-terminating execution. Prior work [59, 64, 131] addresses this using linker namespaces [58], which provide the interposer and the application with separate instances of their shared library dependencies. Additionally, loading a separate copy of `libc` and other libraries into the

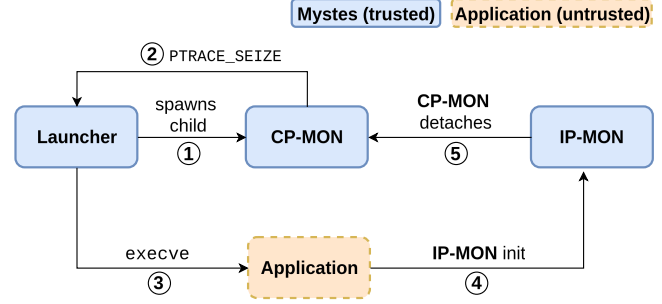


Figure 1. Control flow of Mystes during application startup. Trusted Mystes components are shown in blue (solid border); the untrusted application in orange (dashed border).

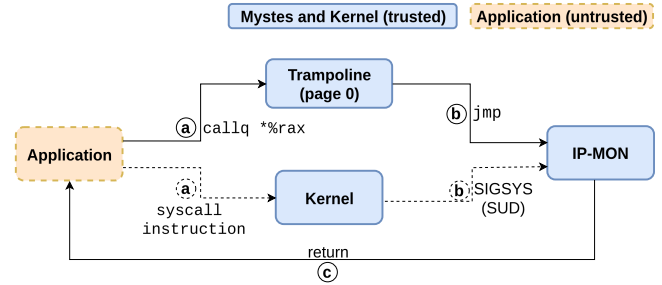


Figure 2. IP-MON’s interposition steps. When an application executes a system call, two paths are possible. The fast path (solid steps): the rewritten `callq %rax` instruction redirects execution to the trampoline at page 0 (installed by IP-MON at initialization) (a), which jumps to IP-MON (b). The slow path (dashed steps): a non-rewritten `syscall` instruction traps to the kernel via SUD (a’), which delivers a `SIGSYS` signal to IP-MON (b’). After handling, IP-MON typically returns control to the application (c). Trusted Mystes components and kernel are shown in blue (solid border); the untrusted application in orange (dashed border).

process causes *gadget inflation* [69], significantly increasing the available code gadgets and expanding the attack surface. To address this, we designed IP-MON as predominantly self-contained code, relying on a small subset of `libc` functions that neither maintain shared state nor contain syscall instructions. Any syscall instructions within IP-MON itself are embedded directly and explicitly excluded from rewriting. Consequently, Mystes does not require linker namespaces.

To ensure that binary rewriting does not introduce *known rewriting issues* [59] that can lead to code or data corruption³, Mystes rewrites only *verified* syscall instructions identified during an offline phase. Unlike K23 [59], which relies on representative inputs, we leverage coverage-guided fuzzing

³*zpoline*’s static analysis can misidentify code or data as syscall instructions and rewrite it, causing corruption; an attacker can abuse *lazypoline* by redirecting control flow to trigger on-the-fly rewriting of arbitrary code or data that resembles syscall instructions [59].

Issue	zpoline [131]	lazypoline [64]	K23 [59]	Mystes
Known Rewriting Issues [59]	Yes	Yes	No	No
Gadget Inflation [69]	Yes	Yes	Yes	No
Free Syscall Gadget	No	Yes	No	No
Test Suite Required	No	No	Yes	No

Table 1. State-of-the-art rewriting-based interposers.

using AFL++ [49] combined with SUD-based logging to automatically discover and record syscall instructions without requiring any test infrastructure. We additionally introduce two heuristics that guide the fuzzer toward inputs more likely to exercise syscall invocation sites (see Section A). Furthermore, unlike prior approaches that treat each application independently, Mystes adopts an *all for one and one for all*⁴ strategy: syscall instruction offsets identified within shared libraries (e.g., `libc.so`) during a fuzzing campaign for one application can be directly reused to rewrite those same instructions in any other application that links against the same library. Consequently, any fuzzing campaign contributes to the verified syscall set of other applications sharing the same libraries. This is particularly effective in practice, as prior work [59] and our own experiments confirm that the vast majority of executed system calls are issued through `libc`. Any syscall instructions not covered by the fuzzing campaign are transparently handled via SUD, ensuring comprehensive mediation regardless of coverage.

Like prior work [59, 64, 131], IP-MON places a trampoline at page zero, where the small non-negative system call numbers used as the call target in `callq %rax` naturally land. Since accesses to the null page would normally trigger a segmentation fault, these approaches use PKU to tag page zero as execute-only, preserving the expected crash behavior for read and write null-pointer accesses. Note that PKU only controls read and write permissions; instruction fetches are unaffected, which is precisely what allows the trampoline to execute. To restrict which calls into the trampoline are valid, zpoline and K23 additionally maintain a bitmap of rewritten syscall instructions, permitting only calls originating from known patched locations. Prior work motivates this primarily for compatibility: existing tools and sanitizers assume that null-pointer dereferences crash the process, and the bitmap preserves that assumption.

However, we identify an additional security motivation: without such a check, an attacker who can redirect control flow to page zero gains a free system call gadget — page zero is always mapped and always reaches the trampoline. Exploiting such a gadget normally requires a prior information-disclosure vulnerability to learn a usable code address; the null page eliminates that requirement entirely.

⁴A nod to the motto of the Three Musketeers [47].

Notably, syscall gadgets represent the final step in many real-world attacks [66, 67, 118, 132]. This concern is particularly acute for on-the-fly rewriting approaches such as lazypoline: beyond the known rewriting issues that can lead to security or compatibility consequences [59], on-the-fly rewriting inherently cannot maintain a bitmap during execution, permanently exposing the null-page trampoline as an attacker-controlled syscall primitive. Mystes therefore maintains a bitmap specifically to close this attack vector. Table 1 illustrates how Mystes compares to prior binary rewriting-based interposers.

5.2 Complete Isolation by Default

Since IP-MON operates within the same address space as the application, Mystes uses PKU to protect all of IP-MON’s state, including its code and data, from a compromised application. IP-MON and the application are placed in disjoint sets of PKU-tagged pages associated with distinct protection keys. To maintain a fully separate execution environment, IP-MON uses its own stack, heap, and a GS-based TLS-like area for per-thread state, all of which are protected using PKU. While the standard Linux ABI reserves the FS register for application thread-local storage, systems such as Endokernel [130] use GS for their own per-thread state, keeping the interposer’s thread-local storage separate from the application’s. This separation protects critical state, such as the SUD selector, synchronization primitives, and other metadata required for correct and secure interposition. We collectively refer to all PKU-protected regions of IP-MON as *trusted memory*. Similar to prior work [106, 115, 119, 130], all control transfers between the application and IP-MON, both on entry and on exit, occur exclusively through well-defined, exploit-resistant instruction sequences, commonly referred to as *call gates*. These gates ensure that every transition correctly switches the execution context.

Beyond isolation, ensuring correct and robust mediation is also challenging. Prior work has shown that safely interposing system calls is non-trivial due to time-of-check-to-time-of-use (TOCTOU) races, particularly in multi-threaded applications [43, 106, 119, 130]. For example, one thread may modify shared state after it has been checked by the interposer but before it is used, leading to inconsistent or unsafe decisions. To mitigate such issues, Mystes copies relevant system call state, such as memory referenced by pointer-based arguments, into trusted memory before performing any checks. This trusted memory, protected using PKU (e.g., the interposer’s stack or heap), ensures that validation and enforcement operate on consistent, tamper-resistant snapshots of system call state. In addition, Mystes follows established practices from prior work [106, 119, 130] to carefully structure checks and state transitions in a race-resistant manner, minimizing the window for concurrent interference.

5.3 Sandbox Design

While PKU provides hardware-enforced isolation of IP-MON’s state, prior work has shown that such isolation can be bypassed by a determined attacker [43, 60, 106, 115, 119, 130]. To prevent this, Mystes employs an in-process sandbox built on top of its own interposition mechanism — effectively, the interposer protects itself from a compromised application.

5.3.1 Preventing Unsafe Instruction-Based Attacks.

Even though PKU enables efficient isolation, its guarantees can be undermined by a class of user-space instructions, commonly referred to as *unsafe instructions*. This attack vector has been highlighted in prior work [60, 115, 119], and systems such as Endokernel [130] and Jenny [106] mitigate it using sandboxing. However, while designing and implementing Mystes’ sandbox, we identified several limitations in these approaches, motivating a more robust design.

Prior work has identified unsafe instructions that can undermine PKU-based isolation or alter interposer state, including `xrstor`, `wrpkru`, `swapgs`, and `TSX` instructions [60, 106, 115, 119, 130]. To the best of our knowledge, we identify a previously overlooked attack vector in multi-threaded interposers: as described above, interposers maintain per-thread state through a GS-based TLS-like area [59, 64, 130, 131], but a malicious application can use `wrgsbase` to redirect the GS base to attacker-controlled memory, effectively forging this area. Consistent with prior observations [115, 119], we find that most unsafe instructions originate from widely used shared libraries (e.g., `libc`). Linker namespaces duplicate these dependencies, multiplying unsafe instruction instances and expanding the attack surface. By avoiding namespaces (see Section 5.1), Mystes avoids this side effect, reducing the number of instructions that must be vetted.

Handling Unsafe Instructions. Similar to prior work [60, 106, 115, 119, 130], Mystes’ sandbox employs binary scanning at runtime to identify unsafe instructions within the application. We also extend this analysis to detect `wrgsbase`, capturing the previously overlooked attack vector described above. Furthermore, we carefully design the scanning process to avoid race conditions during analysis, following best practices established in prior work [106, 119, 130]. After identifying unsafe instructions, Endokernel [130] and Jenny [106] neutralize them via static binary instrumentation (SBI) or by marking the corresponding pages non-executable. Both have limitations: prior work [119] shows that SBI cannot reliably eliminate all unsafe instructions, due to the inherent difficulty of distinguishing code from data [125] and gaps in rewriting rules [1], while marking pages non-executable can break legitimate application behavior. Cerberus [119] and Hodor [60] instead use hardware breakpoints, but Cerberus relies on `ptrace` and Hodor requires kernel modifications — neither of which Mystes can rely on, since IP-MON runs entirely in user space and `ptrace` would incur significant overhead. To address this, Mystes’

sandbox instead uses `perf_event_open` [15] to configure hardware breakpoints from user space.

Enforcing the W^X Invariant. Similar to prior work [106, 119, 130], Mystes’ sandbox enforces a strict W^X policy over all memory regions to prevent attackers from injecting or modifying executable code at runtime. In particular, the sandbox intercepts memory-management system calls that create or modify mappings, such as `mmap`, `mprotect`, and `mremap`, and ensures that any page marked as executable is not simultaneously writable. Since commodity Linux systems do not enforce this invariant by default, this check is essential to prevent modification of executable regions through writable memory. Furthermore, the sandbox restricts the creation of executable shared mappings. Shared memory regions (e.g., `MAP_SHARED`) can be modified outside the control of the interposition logic, potentially bypassing monitoring and violating isolation guarantees. To mitigate this risk, the sandbox disallows or tightly constrains executable shared mappings, ensuring that all executable code remains under the interposer’s control.

Enforcing W^X for file-backed mappings is challenging, as the underlying file may be modified after mapping, potentially altering executable code without the interposer’s knowledge [43]. Prior work explores several approaches [106, 119, 130]. One option is to disallow mappings of files writable by the application, which requires monitoring system calls that affect file permissions (e.g., `chmod`, `chown`). Another approach enforces immutability by removing write permissions or creating a protected copy. A third option replaces file-backed mappings with anonymous ones (e.g., `MAP_ANONYMOUS`) and copies file contents into memory, eliminating the dependency on the underlying file. However, this comes at the cost of losing page cache benefits and increasing memory overhead due to duplicated contents.

Mystes’ sandbox, similar to prior work [106], disallows executable mappings of files writable by the application and monitors system calls that modify file permissions. Unlike approaches that copy file contents into anonymous mappings, this preserves page cache sharing across processes and avoids the associated memory overhead. However, we observe that executable code can also be introduced via program loading calls (e.g., `execve`, `execveat`), which prior work entirely overlooks [106, 119, 130], leaving gaps in W^X enforcement. *The sandbox additionally mediates these calls to ensure that executable regions are not mapped from writable files throughout the application lifetime.*

5.3.2 Preventing Interposition Interference via System Interfaces.

Prior work has extensively studied attacks that abuse system interfaces to interfere with system call interposition, including circumventing page-table and PKU protections and disabling interposition altogether [106, 115, 119, 130]. Building on these insights, Mystes’ sandbox mediates system calls affecting process state or memory protection, to

address interposition interference. For example, it restricts access to interfaces such as `/proc/self/mem`, which directly accesses process memory while bypassing memory protections, and mediates calls that can weaken or disable interposition (e.g., `prctl`, `execve`). Inspired by Endokernel [130], Mystes’ sandbox also provides full signal virtualization: the kernel resets PKRU on signal delivery without awareness of the monitor’s domain, potentially exposing trusted memory to the application’s handler or enabling `sigreturn` to be abused to forge arbitrary PKRU values. To this end, the sandbox intercepts signal registration calls and maintains its own delivery mechanism, ensuring signals are always delivered at safe execution points with the correct PKU context.

Pointer Validation. Although *ordinary* system calls cannot normally bypass page-table or PKU-based memory protections, pointer-bearing system calls originating from the untrusted application can still be abused as confused deputies during interposition. Once a system call is intercepted, the interposer typically executes within its own privileged execution context, where trusted memory protections are temporarily relaxed to permit access to interposer state. Moreover, the interposer commonly executes the system call itself on behalf of the untrusted application. Consequently, kernel accesses performed during system call handling may inadvertently read from or write to trusted memory unless pointer arguments are carefully validated. Prior work [106, 130] addresses this by checking the ranges of pointer arguments and validating them against trusted memory regions, but only for common data-transfer interfaces such as `read` and `write`, leaving the broader system call surface uncovered. Less obvious interfaces, including `clone`, `accept`, `select`, `pipe`, and related system calls that read from or write to user-provided memory, are left unprotected. Mystes’ sandbox addresses this by executing application system calls with the PKRU register configured to permit access only to untrusted memory. As a result, any kernel access to trusted memory through a pointer argument results in an `EFAULT` error.

Performance Events Protection. Mystes’ sandbox introduces additional attack surface due to its use of hardware breakpoints configured via `perf_event_open`. It relies on performance events to vet unsafe instructions, exposing file descriptors that a malicious application may attempt to manipulate. To mitigate this, the sandbox mediates system calls that directly affect file descriptor state (e.g., `close`, `close_range`, `dup`, `ioctl`), ensuring that these descriptors cannot be closed, reused, or altered. Additionally, file descriptors can be covertly transferred to other processes via `sendmsg` with `SCM_RIGHTS` ancillary data, or duplicated from another process via `pidfd_getfd`, bypassing the above protections. The sandbox defends against this by inspecting the `msg_level` and `msg_type` fields and walking the full `cmsghdr` chain in `sendmsg` and `recvmsg` calls to detect and reject entries referencing protected descriptors, and by mediating `pidfd_getfd`. Finally, when the application calls `fork`

or `clone`, the child process inherits all open file descriptors, including those associated with performance events. Since the underlying hardware breakpoint resource is shared rather than copied across `fork`, manipulating or closing these descriptors in the child also affects the parent’s monitoring. The sandbox addresses this by mediating `fork/clone` to establish fresh, independent performance events in the child.

6 Implementation

We implement a Linux x86-64 prototype of Mystes, consisting of ~10,500 LoC, primarily in C and C++, with additional assembly and build scripts. Mystes employs four PKU keys to enforce isolation within the shared address space: one for application state accessible by both the application and Mystes, one for Mystes’ state that is inaccessible to the application, one for state that is read-only to the application but fully accessible by Mystes (e.g., the SUD selector), and one for eExecute-Only Memory (XOM), e.g., the trampoline page at virtual address 0. Mystes uses `pkey_mprotect` to tag memory regions with the appropriate keys.

7 Evaluation

We evaluate Mystes along two dimensions: security and performance. First, we assess its security guarantees by analyzing its resilience against all known attacks identified by prior work on PKU-based isolation, as well as newly identified attack vectors discovered during the design and development of Mystes (see Sections 5.3.1 and 5.3.2). Second, we evaluate the performance impact of Mystes using microbenchmarks and real-world system call-intensive workloads.

7.1 Security Evaluation

We reviewed prior work on PKU-based isolation and in-process system call interposition and audited the x86-64 system call interface of the Linux kernel used in our evaluation, examining interfaces affecting memory isolation, executable mappings, process/signal state, or the protected resources of Mystes. For attacks targeting the in-process interposer or its PKU-based isolation layer, we compare Mystes against the state-of-the-art PKU-based systems Endokernel [130] and Jenny [106], summarizing results in Table 2. For each identified candidate, we adapted existing or implemented new proof-of-concept exploits. Beyond leaving security gaps, neither system can be integrated into Mystes, since they require intrusive changes (see Section 4): Endokernel requires a custom `libc/libOS`, and Jenny requires kernel modifications. Mystes prevents all identified attacks, whereas prior systems exhibit security gaps that an attacker can exploit. Separately, the security issues described in Section 5.1 (see also Table 1) — namely *Known Rewriting Issues*, *Gadget Inflation*, and *Free Syscall Gadget* — are unique to binary rewriting approaches; Mystes addresses all three by design.

Attack	Mystes	Endokernel [130]	Jenny [106]
Inconsistency of PT Permissions [43]	•	•	•
Mutable Mappings [43]	•	•	•
Changing Code by Relocation [43]	•	•	•
Modifying PKRU via sigreturn [43]	•	•	•
Race Condition in Scanning [43]	•	•	•
Determination of Trusted Mappings [43]	•	•	•
Influencing with seccomp [43]	•	•	•
Modifying Trusted Mappings [43]	•	•	•
Syscall TOCTOU Attack [106, 130]	•	•	•
Page Table Syscalls Abuse [130]	•	•	○
rseq Control Flow Hijacks [130]	•	•	○
Forged Signal Delivery [130]	•	•	•
Incorrect Signal Return Handling [130]	•	•	○
Inconsistency of Monitor and Kernel Status [130]	•	•	○
Vetted Unsafe Instruction Relocation [119]	•	–	–
Incomplete Debug Register Update [119]	•	–	–
Incomplete Unsafe Instruction Handling (new attack, Section 5.3.1)	•	○	○
Incomplete W*X Enforcement (new attack, Section 5.3.1)	•	○	○
Incomplete Pointer Validation (new attack, Section 5.3.2)	•	○	○
Performance Events Protection (new attack, Section 5.3.2)	•	–	–

Table 2. Comparative security analysis based on known and new attacks. ○ = vulnerable, • = prevented, – = not applicable.

7.2 Performance Evaluation

Unless stated otherwise, all experiments were performed on a Dell Precision 7960 workstation with a 12-core Intel Xeon w5-3425 CPU at 3.20 GHz, 64 GB of RAM, and a 500 GB NVMe SSD. To reduce measurement variability, we disabled Turbo Boost, Hyper-Threading, and CPU frequency scaling [33]. The system ran Ubuntu 24.04.4 LTS with Linux kernel 7.0.0-28 and glibc 2.39. Each experiment was repeated 10 times. We discarded the highest and lowest measurements as outliers and report the geometric mean overhead relative to a native baseline. For all experiments, we use our collaborative fuzzing approach to construct the offline list of verified syscall instructions (see Section 5.1). Each fuzzing campaign saturates within at most one hour. All benchmarks are designed to stress interposers under system call-intensive conditions, and we ensure that the relevant CPU cores are fully saturated to expose performance bottlenecks. Our evaluation setup is consistent with relevant prior work [5, 59, 115, 119, 130, 131]. Throughout this section, the SUD configuration refers to Mystes with binary rewriting disabled, relying solely on the SUD-based slow path.

7.2.1 Microbenchmarks. We use lmbench [91], a standard benchmarking suite, to assess the overhead of our interposition mechanism on commonly used system calls. Specifically, we use the following lmbench tests: null, open, read, and write. We compare Mystes against zpoline [131], which is, to the best of our knowledge, the fastest binary rewriting-based system call interposer, and SUD. The results are shown in Figure 3. As expected, SUD incurs the highest

overhead, as mode-switching overhead dominates especially for the null test, which mainly executes lightweight system calls with minimal kernel-side work. zpoline achieves near-native performance, demonstrating the efficiency of rewriting-based interposition without security enforcement. Mystes incurs a modest overhead over zpoline, stemming from PKU-based isolation, trusted memory management, and additional checks in the interposition path. These costs are constant per system call and therefore most prominent in microbenchmarks, while being largely amortized in realistic workloads, as confirmed by our macrobenchmarks.

7.2.2 Case Study 1: Adaptive System Call Filtering.

System call filtering confines a process to only the system calls required for its benign execution, reducing the attack surface available to an adversary who gains control of the process. seccomp-bpf is the standard Linux mechanism for this, but its hierarchical design forces a process to allow all system calls needed by any program it may subsequently execve, leading to over-privilege. While the hierarchical design correctly ensures that a child process cannot exceed the parent’s allowed set, it forces the parent to pre-authorize all system calls that any descendant may require — even system calls the parent never invokes during its own execution. A compromised parent can thus invoke these extra system calls as part of an attack, despite never legitimately needing them. Prior work shows this can result in allowing up to 113.73% more system calls than a program actually needs [51]. Addressing this over-privilege adaptively, so that each program receives its own least-privilege policy rather than inheriting

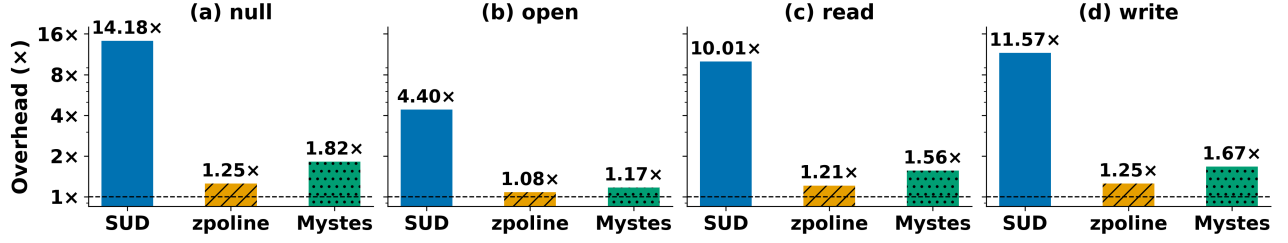


Figure 3. System call overhead for lmbench. Standard deviation is below 0.83% in all cases.

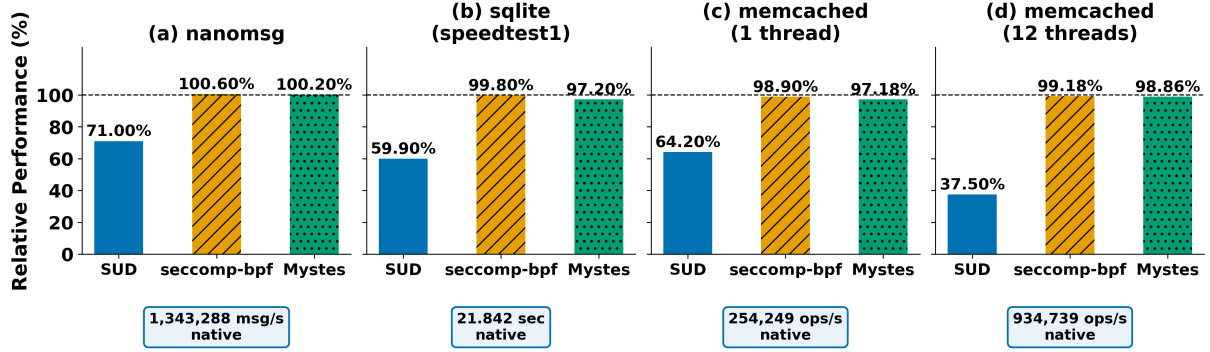


Figure 4. Case Study 1. We report native performance (msg/s, seconds, or ops/s) and relative performance under SUD, seccomp-bpf, and Mystes. Standard deviation is below 1.8% in all cases.

its parent’s, has so far required kernel modifications [51]. Mystes instead enables per-program adaptive system call filtering entirely in user space, without any kernel changes: each program installs its own tight, least-privilege policy at launch, independently of subsequently executed components. We implement filtering as a bitmap of allowed system calls initialized in trusted memory, enabling constant-time checks with minimal overhead. For this case study, we use Sysfilter [44] to generate each application’s filtering policy; our approach is agnostic to the policy generation tool and can integrate with others as well [40, 55, 56, 104].

For this case study, we use three popular applications: nanomsg [12], sqlite [20], and memcached [10]. Each application is executed either natively, under Mystes with adaptive system call filtering enabled, under seccomp-bpf, the standard kernel mechanism for system call filtering, or under SUD. Client workloads run natively and drive each server. To benchmark nanomsg, we use the local throughput benchmark with a receiver (local_thr) and sender (remote_thr) communicating over a loopback TCP endpoint, sending 2M messages of 1 KB each and measuring throughput in msg/s. For sqlite, we evaluate a single configuration using a fresh database with 4 KiB pages, WAL journaling, synchronous=NORMAL, and no auto-checkpointing, using the upstream speedtest1 benchmark with -size=800 and reporting execution time. For memcached, we use memtier_benchmark with 14 client threads, 50 clients, a 30-second duration, read-only requests

(-ratio=1:0), random key access, and 128-byte values, evaluating configurations with 1 and 12 worker threads and measuring throughput in ops/s.

We present our results in Figure 4. Across all workloads, Mystes and seccomp-bpf achieve near-native performance, while SUD incurs substantial overhead across all workloads. Mystes imposes at most a few percentage points of overhead relative to seccomp-bpf. SUD’s overhead is not uniform across workloads: it is most severe for memcached, where relative throughput drops from 64.2% with a single worker thread to 37.5% with 12.

These results show that Mystes matches seccomp-bpf’s near-native performance while additionally providing strictly stronger, per-program filtering, since its constant-time bitmap check executes directly within the already-redirected, in-process syscall path, adding negligible marginal cost on top of the baseline rewriting overhead. SUD’s substantial overhead, in contrast, confirms that signal-based mode switching is prohibitive for system call-intensive workloads; its degradation on memcached as thread count grows indicates that the fixed per-syscall cost of signal-based mode switching compounds as concurrent threads independently incur it.

7.2.3 Case Study 2: Enhanced XOM. eXecute-Only Memory (XOM) prevents code pages from being read while still allowing execution, mitigating attacks that rely on disclosing executable code [35, 111]. Prior work [86, 135] implements XOM using PKU, but in isolation such schemes offer limited

Workload	Native (req/s)	Enhanced XOM-Switch (SUD) (rel. %)	Enhanced XOM-Switch (Mystes) (rel. %)
nginx (1 worker)	160,690	42.8	93.0
nginx (10 workers)	1,091,144	44.5	94.7
lighttpd (1 worker)	156,372	58.8	95.3
lighttpd (10 workers)	1,217,346	56.4	95.6

Table 3. Case Study 2. Performance across popular servers for a representative workload (4 KB files). We report native throughput (req/s) and relative performance. Standard deviation is below 0.33% in all cases.

protection, as attackers with control-flow capabilities can disable or bypass XOM. Mystes enables practical enforcement of XOM entirely in user space by extending its trusted memory tracking to XOM pages, protecting them from disclosure. We use Mystes to protect XOM-Switch [135], a tool that automatically applies PKU-based XOM to binaries, and refer to the combination as Enhanced XOM-Switch.

For this case study, we use two popular web servers: nginx [13] and lighttpd [7]. For benchmarking, we use static-file serving workloads driven by wrk over a loop-back TCP connection. The web server process is executed either natively, under Enhanced XOM-Switch (SUD), or under Enhanced XOM-Switch (Mystes), while wrk runs natively and generates client requests. We evaluate both single-worker and 10-worker server configurations serving a static 4 KB file. For each configuration, we match the number of wrk threads to the number of server workers, use 16 connections per client thread, and run each benchmark for 30 seconds. Note that serving a small, cached file represents an extreme test case, as the server performs minimal application-level work [115]. To obtain a more realistic assessment, we additionally evaluate larger files up to 64 KB for the single-worker configuration. We note that XOM-Switch alone introduces no noticeable overhead in our experiments and we therefore do not report results for that configuration.

We present our results in Tables 3 and 4. The overhead of Enhanced XOM-Switch (Mystes) is most visible only in extreme workloads dominated almost entirely by system call processing, such as serving very small cached files, and rapidly diminishes as file sizes increase, becoming effectively negligible for realistic workloads (e.g., 32–64 KB). Enhanced XOM-Switch (SUD) incurs significant overhead across nearly all workloads, narrowing only for the largest files (e.g., nginx at 64 KB). Scaling from 1 to 10 workers leaves SUD’s relative overhead essentially unchanged for both nginx (42.8% to 44.5%) and lighttpd (58.8% to 56.4%). At the largest tested file sizes, Enhanced XOM-Switch (Mystes) reaches parity with, or marginally exceeds, native throughput for both servers.

Workload	Native (req/s)	Enhanced XOM-Switch (SUD) (rel. %)	Enhanced XOM-Switch (Mystes) (rel. %)
nginx (0 KB)	196,731	49.5	92.8
nginx (4 KB)	160,690	42.8	93.0
nginx (16 KB)	161,327	43.0	95.9
nginx (32 KB)	147,985	45.4	100.2
nginx (64 KB)	59,873	94.0	99.6
lighttpd (0 KB)	184,665	60.1	95.3
lighttpd (4 KB)	156,372	58.8	95.3
lighttpd (16 KB)	152,469	50.8	95.4
lighttpd (32 KB)	148,490	52.3	96.4
lighttpd (64 KB)	94,800	72.4	100.3

Table 4. Case Study 2. Impact of file size on throughput for nginx and lighttpd (single-worker). We report native throughput (req/s) and relative performance. Standard deviation is below 1.2% in all cases.

These results indicate that Mystes provides fast and secure system call mediation. This efficiency stems directly from how Mystes implements XOM protection: rather than introducing a separate enforcement mechanism, it simply extends the same trusted memory tracking already used to isolate IP-MON’s own state (see Section 5.2) to also cover XOM pages, so protecting XOM-Switch’s pages adds minimal cost beyond Mystes’ baseline interposition and sandboxing overhead. Once network and I/O processing dominate execution time, as with larger file sizes, this overhead becomes effectively negligible and, in the largest configurations, fully hidden. In contrast, Enhanced XOM-Switch (SUD)’s persistent overhead confirms that signal-based mode switching is prohibitive for system call-intensive workloads. Comparing Table 3 to Case Study 1 also reveals a contrast in how concurrency affects SUD’s overhead: unlike the sharp degradation observed for memcached’s threads in Case Study 1, scaling from 1 to 10 workers leaves SUD’s relative overhead here essentially unchanged for both nginx and lighttpd.

7.2.4 Case Study 3: Transparently Integrating a User-Space Network Stack. User-space network stacks such as lwIP [48] backed by kernel-bypass frameworks such as the Data Plane Development Kit (DPDK) [8] achieve significantly higher networking performance than the kernel TCP/IP stack by eliminating costly kernel transitions on the data path. However, their adoption is hindered by a fundamental usability barrier: applications must be explicitly modified to use them, as they expose a different API from the standard POSIX socket interface. System call interposition offers a principled solution to this problem: by transparently redirecting socket-related system calls to the user-space stack,

Mechanism	Throughput (req/s)
Kernel TCP/IP (native)	172,876
SUD	165,788
LD_PRELOAD	917,972
zpoline	870,421
Mystes	856,494

Table 5. Case Study 3. Redis throughput using either the native Linux TCP/IP stack or lwIP/DPDK. We integrate lwIP/DPDK transparently using LD_PRELOAD-based function hooking or the evaluated system call interposition mechanisms. Standard deviation is below 0.71% in all cases.

unmodified applications can benefit from kernel bypass without any source-level changes. zpoline [131] demonstrated this use case, showing that its low-overhead binary rewriting enables performance exceeding that of the kernel TCP/IP stack when transparently bonding a user-space TCP/IP stack to existing applications.

Mystes enables the same transparent integration while additionally protecting the interposition logic from a compromised application. Similarly to zpoline, we evaluate this use case using Redis [17]. We use two machines of the same hardware configuration as described in Section 7.2, directly connected via Mellanox ConnectX-5 100 Gbps NICs; one machine acts as the server and the other as the client. The Redis server is executed under the evaluated mechanism on the server machine, while redis-benchmark runs natively on the client machine. We use redis-benchmark, distributed as part of the Redis source, with a GET-only workload, similar to the zpoline paper [131], so that the server spends most of its time on networking operations rather than disk operations. Requests are sent over 32 persistent concurrent connections, and we measure throughput in req/s.

We present our results in Table 5, comparing Mystes against zpoline, SUD, and LD_PRELOAD, using the kernel TCP/IP stack as the native baseline. For the LD_PRELOAD baseline, we use LD_PRELOAD-based function hooking⁵ instead of system call interposition to transparently redirect socket-related library calls to lwIP. LD_PRELOAD, zpoline, and Mystes all substantially outperform the kernel TCP/IP stack, with LD_PRELOAD achieving 5.31× the native throughput. zpoline reaches 94.8% of LD_PRELOAD throughput, while Mystes reaches 93.3%. SUD reaches only 95.9% of the kernel TCP/IP baseline. Mystes’ throughput is only 1.6% below zpoline’s (856,494 vs. 870,421 req/s).

These results demonstrate that Mystes’ additional security guarantees come at a very low cost: even though zpoline

⁵Prior work has shown that LD_PRELOAD-based function hooking should not be used in systems requiring reliability or security [131]. Mystes uses LD_PRELOAD solely to inject IP-MON into the process, not for function hooking. Alternatively, other tools such as patchelf [14] can be used for this injection.

enforces no security policy at all, Mystes trails it by only 1.6%, showing that PKU-based isolation and sandboxing add negligible overhead even under a demanding, I/O-bound network workload. SUD’s comparatively poor result instead reflects its signal-based interposition, which adds overhead on every network system call. In absolute terms, Mystes still sustains nearly 5× the throughput of the kernel TCP/IP stack, whereas SUD, lacking binary rewriting, cannot even match that baseline, underscoring that low-overhead binary rewriting, not interposition alone, is what unlocks the performance benefits of bypassing the kernel network stack.

8 Discussion and Related Work

Our current design and implementation focus on x86-64, which remains the dominant architecture in server and cloud deployments. Extending Mystes to other architectures is feasible, as our approach is largely orthogonal to the underlying instruction set and can leverage existing binary rewriting techniques [31, 93, 109]. Mystes relies on PKU to provide efficient in-process isolation. While PKU is widely supported in modern x86-64 CPUs, our design is not fundamentally tied to it. Alternative mechanisms can be used to achieve similar isolation guarantees [28, 30, 46, 71, 75, 78–81, 84, 90, 102, 113, 114, 124, 126–128].

Alternative interposition approaches include intrusive techniques that rely on hardware or OS modifications [3, 32, 34, 38, 52, 60, 74, 85, 95, 115, 119, 122, 136]. While capable of high performance, they are difficult to maintain, reduce flexibility, and increase the trusted computing base (TCB). Another class relies on function hooking, interposing wrapper functions instead of system calls [41, 123, 133]. These approaches are lightweight but cannot capture system calls issued outside wrappers and limit visibility to function-level arguments and return values. Moreover, identifying and mapping all relevant wrappers is non-trivial [31, 59, 64, 131].

9 Conclusion

We presented Mystes, the first practical system call interposition approach that achieves strong security and high performance. We extensively evaluated the security of Mystes against all known attacks identified by prior work as well as newly discovered attack vectors, and assessed its performance on both microbenchmarks and real-world system call-intensive workloads. Our results show that Mystes successfully prevents all identified attacks while closely approximating the performance of non-security-focused interposers.

AI Use Disclosure

AI-based tools (Claude and Codex) were used for language refinement, grammar checking, and limited drafting and coding assistance. All AI-assisted content was reviewed and validated by the authors, who take full responsibility for the manuscript’s technical content and conclusions.

References

- [1] 2021. Unintended Instructions on x86. Philip Reames – A collection of (public) notes on assorted topics. <https://github.com/preames/public-notes/blob/master/unintended-instructions.rst>
- [2] 2026. Intel. Intel 64 and IA-32 Architectures Software Developer Manuals.
- [3] Last accessed 2026. falco. <https://falco.org/>
- [4] Last accessed 2026. gdb - The GNU Debugger. <https://man7.org/linux/man-pages/man1/gdb.1.html>
- [5] Last accessed 2026. Jenny. <https://github.com/isec-tugraz/Jenny>
- [6] Last accessed 2026. ld.so, ld-linux.so – dynamic linker/loader. <https://man7.org/linux/man-pages/man8/ld.so.8.html>
- [7] Last accessed 2026. lighttpd. <https://www.lighttpd.net/>
- [8] Last accessed 2026. Linux Foundation. Data plane development kit (DPDK).
- [9] Last accessed 2026. ltrace - A library call tracer. <https://man7.org/linux/man-pages/man1/ltrace.1.html>
- [10] Last accessed 2026. memcached. <https://memcached.org/>
- [11] Last accessed 2026. mprotect, pkey_mprotect – set protection on a region of memory. https://man7.org/linux/man-pages/man2/pkey_mprotect.2.html
- [12] Last accessed 2026. nanomsg. <https://nanomsg.org/>
- [13] Last accessed 2026. nginx. <https://nginx.org/>
- [14] Last accessed 2026. patchelf: A small utility to modify the dynamic linker and RPATH of ELF executables. <https://github.com/NixOS/patchelf>
- [15] Last accessed 2026. perf_event_open(2) - Linux manual page. https://man7.org/linux/man-pages/man2/perf_event_open.2.html
- [16] Last accessed 2026. ptrace - process trace. <http://man7.org/linux/man-pages/man2/ptrace.2.html>
- [17] Last accessed 2026. redis. <https://redis.io/>
- [18] Last accessed 2026. rr. <https://github.com/rr-debugger/rr>
- [19] Last accessed 2026. seccomp - operate on Secure Computing state of the process. <https://man7.org/linux/man-pages/man2/seccomp.2.html>
- [20] Last accessed 2026. sqlite. <https://sqlite.org/>
- [21] Last accessed 2026. Steam. <https://store.steampowered.com/>
- [22] Last accessed 2026. strace - trace system calls and signals. <https://man7.org/linux/man-pages/man1/strace.1.html>
- [23] Last accessed 2026. Syscall User Dispatch. <https://www.kernel.org/doc/html/latest/admin-guide/syscall-user-dispatch.html>
- [24] Last accessed 2026. syscall_intercept. https://github.com/pmem/syscall_intercept
- [25] Last accessed 2026. Wine. <https://www.winehq.org/>
- [26] Martín Abadi, Mihai Budiu, Ulfar Erlingsson, and Jay Ligatti. 2005. Control-Flow Integrity. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [27] Karan Aggarwal, Chenlei Zhang, Joshua Charles Campbell, Abram Hindle, and Eleni Stroulia. 2014. The power of system call traces: predicting the software energy consumption impact of changes. In *Proceedings of the Annual International Conference on Computer Science and Software Engineering (CASCON)*.
- [28] Saar Amar, Tony Chen, David Chisnall, Nathaniel Wesley Filardo, Ben Laurie, Hugo Lefeuvre, Kunyan Liu, Simon W. Moore, Robert Norton-Wright, Margo Seltzer, Yucong Tao, Robert N. M. Watson, and Hongyan Xia. 2025. CHERIoT RTOS: An OS for Fine-Grained Memory-Safe Compartments on Low-Cost Embedded Devices. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.
- [29] Dennis Andriesse, Xi Chen, Victor van der Veen, Asia Slowinska, and Herbert Bos. 2016. An In-Depth Analysis of Disassembly on Full-Scale x86/x64 Binaries. In *Proceedings of the USENIX Security Symposium*.
- [30] ARM. Last accessed 2026. Permission Overlays. <https://developer.arm.com/documentation/102376/0200/Permission-indirection-and-permission-overlay-extensions/Permission-overlays>
- [31] Paul-Antoine Arras, Anastasios Andronidis, Luis Pina, Karolis Mituzas, Qianyi Shu, Daniel Grumberg, and Cristian Cadar. 2022. SaBRE: Load-time Selective Binary Rewriting. *International Journal on Software Tools for Technology Transfer* (2022).
- [32] Adam Belay, Andrea Bittau, Ali Mashtizadeh, David Terei, David Mazières, and Christos Kozyrakis. 2012. Dune: Safe User-Level Access to Privileged CPU Features. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [33] Dirk Beyer, Stefan Löwe, and Philipp Wendler. 2019. Reliable benchmarking: requirements and solutions. *Int. J. Softw. Tools Technol. Transf.* 21, 1 (Feb. 2019), 1–29. doi:10.1007/s10009-017-0469-y
- [34] Atri Bhattacharyya, Florian Hofhammer, Yuanlong Li, Siddharth Gupta, Andres Sanchez, Babak Falsafi, and Mathias Payer. 2023. Securecells: A secure compartmentalized architecture. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [35] Andrea Bittau, Adam Belay, Ali José Mashtizadeh, David Mazières, and Dan Boneh. 2014. Hacking Blind. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [36] Ferdinand Brasser, Lucas Davi, David Gens, Christopher Liebchen, and Ahmad-Reza Sadeghi. 2017. CAN't touch this: Software-only mitigation against Rowhammer attacks targeting kernel memory. In *Proceedings of the USENIX Security Symposium*.
- [37] Danilo Bruschi, Lorenzo Cavallaro, and Andrea Lanzi. 2007. Diversified process replicas for defeating memory error exploits. In *IEEE Performance, Computing, and Communications Conference (IPCCC)*.
- [38] Quinn Burke, Ryan Sheatsley, Yohan Beugin, Eric Pauley, Owen Hines, Michael Swift, and Patrick McDaniel. 2025. Efficient Storage Integrity in Adversarial Settings. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [39] Nathan Burow, Scott A Carr, Joseph Nash, Per Larsen, Michael Franz, Stefan Brunthaler, and Mathias Payer. 2017. Control-flow integrity: Precision, security, and performance. *ACM Computing Surveys (CSUR)* 50, 1 (2017), 16.
- [40] Claudio Canella, Mario Werner, Daniel Gruss, and Michael Schwarz. 2021. Automating Seccomp Filter Generation for Linux Applications. In *Proceedings of the 2021 on Cloud Computing Security Workshop*.
- [41] Buddhika Chamith, Bo Joel Svensson, Luke Dalessandro, and Ryan R. Newton. 2017. Instruction Punning: Lightweight Instrumentation for X86-64. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*.
- [42] Shaiful Alam Chowdhury, Luke N Kumar, Md. Toukir Imam, Mohamed Shazan Mohamed Jabbar, Varun Sapra, Karan Aggarwal, Abram Hindle, and Russell Greiner. 2015. A system-call based model of software energy consumption without hardware instrumentation. In *Proceedings of the International Green and Sustainable Computing Conference (IGSC)*.
- [43] Emma Connor, Tyler McDaniel, Jared M. Smith, and Max Schuchard. 2020. PKU Pitfalls: Attacks on PKU-based Memory Isolation Systems. In *Proceedings of the USENIX Security Symposium*.
- [44] Nicholas DeMarinis, Kent Williams-King, Di Jin, Rodrigo Fonseca, and Vasileios P. Kemerlis. 2020. sysfilter: Automated System Call Filtering for Commodity Software. In *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*.
- [45] Jeff Dike. 2001. User-Mode Linux. In *Annual Linux Showcase & Conference (ALS)*.
- [46] Kha Dinh Duy, Kyuwon Cho, Taehyun Noh, and Hojoon Lee. 2023. Capacity: Cryptographically-Enforced In-Process Capabilities for Modern ARM Architectures. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [47] Alexandre Dumas. 1844. *The Three Musketeers*. Baudry.

- [48] Adam Dunkels. 2001. *Design and Implementation of the lwIP TCP/IP Stack*. Technical Report 2(77). Swedish Institute of Computer Science.
- [49] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Thorsten Holz. 2020. AFL++: Combining Incremental Steps of Fuzzing Research. In *USENIX Workshop on Offensive Technologies (WOOT)*.
- [50] Joshua Fried, Gohar Irfan Chaudhry, Enrique Saurez, Esha Choukse, Inigo Goiri, Sameh Elnikety, Rodrigo Fonseca, and Adam Belay. 2024. Making Kernel Bypass Practical for the Cloud with Junction. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.
- [51] Alexander J. Gaidis, Vaggelis Atlidakis, and Vasileios P. Kemerlis. 2023. SysXCHG: Refining Privilege with Adaptive System Call Filters. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [52] Tal Garfinkel. 2003. Traps and pitfalls: Practical problems in system call interposition based security tools. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.
- [53] Tal Garfinkel, Ben Pfaff, and Mendel Rosenblum. 2004. Ostia: A Delegating Architecture for Secure System Call Interposition. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.
- [54] Stefanos Georgiou and Diomidis Spinellis. 2020. Energy-delay investigation of remote inter-process communication technologies. *Journal of Systems and Software* 162 (2020), 110506.
- [55] Seyedhamed Ghavamnia, Tapti Palit, Azzedine Benameur, and Michalis Polychronakis. 2020. Confine: Automated system call policy generation for container attack surface reduction. In *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*.
- [56] Seyedhamed Ghavamnia, Tapti Palit, Shachee Mishra, and Michalis Polychronakis. 2020. Temporal System Call Specialization for Attack Surface Reduction. In *Proceedings of the USENIX Security Symposium*.
- [57] Adrien Ghosn, Marios Kogias, Mathias Payer, James R. Larus, and Edouard Bugnion. 2021. Enclosure: Language-Based Restriction of Untrusted Libraries. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [58] GNU C Library (glibc). Last accessed 2026. Linker Namespaces - glibc Wiki. <https://sourceware.org/glibc/wiki/LinkerNamespaces>
- [59] Jesús María Gómez, Vissarion Moutafis, Antreas Dionysiou, Fernando Kuipers, Georgios Smaragdakis, Bart Coppens, and Alexios Voulimeas. 2025. Clair Obscur: The Light and Shadow of System Call Interposition: From Pitfalls to Solutions with K23. In *Proceedings of the ACM/IFIP International Middleware Conference*.
- [60] Mohammad Hedayati, Spyridoula Gravani, Ethan Johnson, John Criswell, Michael L. Scott, Kai Shen, and Mike Marty. 2019. Hodor: Intra-Process Isolation for High-Throughput Data Plane Libraries. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [61] Petr Hosek and Cristian Cadar. 2013. Safe Software Updates via Multi-Version Execution. In *Proceedings of the International Conference on Software Engineering (ICSE)*.
- [62] Petr Hosek and Cristian Cadar. 2015. Varan the Unbelievable: An Efficient N-Version Execution Framework. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [63] Yu Charlie Hu and Abhinav Pathak. 2018. System and method for energy usage accounting in software applications. US Patent 10,013,511.
- [64] Adriaan Jacobs, Merve Gülmez, Alicia Andries, Stijn Volckaert, and Alexios Voulimeas. 2024. System Call Interposition Without Compromise. In *Proceedings of the IEEE/IFIP Conference on Dependable Systems and Networks (DSN)*.
- [65] Rob Jansen, Jim Newsome, and Ryan Wails. 2022. Co-opting Linux Processes for High-Performance Network Simulation. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [66] Christopher Jelesnianski, Mohannad Ismail, Yeongjin Jang, Dan Williams, and Changwoo Min. 2023. Protect the System Call, Protect (Most of) the World with BASTION. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [67] Brian Johannesmeyer, Asia Slowinska, Herbert Bos, and Cristiano Giuffrida. 2024. Practical {data-only} attack generation. In *Proceedings of the USENIX Security Symposium*.
- [68] Joonkyo Jung, Jisoo Jang, Yongwan Jo, Jonas Vinck, Alexios Voulimeas, Stijn Volckaert, and Dokyung Song. 2025. Moneta: Ex-Vivo GPU Driver Fuzzing by Recalling In-Vivo Execution States. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.
- [69] Stephen Kell. 2026. System call instrumentation on Linux/x86-64 using memory-indirect calls (in vain?), part one. <https://www.humprog.org/~stephen/blog/2026/06/15/>
- [70] Zijo Kenjar, Tommaso Frassetto, David Gens, Michael Franz, and Ahmad-Reza Sadeghi. 2020. V0ltpwn: Attacking x86 processor integrity from software. In *Proceedings of the USENIX Security Symposium*.
- [71] Juhhee Kim, Jinbum Park, Yoochan Lee, Chengyu Song, Taesoo Kim, and Byoungyoung Lee. 2024. PeTAL: Ensuring Access Control Integrity against Data-only Attacks on Linux. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [72] Taesoo Kim and Nickolai Zeldovich. 2013. Practical and Effective Sandboxing for Non-Root Users. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [73] Paul Kocher, Jann Horn, Anders Fogh, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. 2019. Spectre attacks: Exploiting speculative execution. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [74] Koen Koning, Herbert Bos, and Cristiano Giuffrida. 2016. Secure and efficient multi-variant execution using hardware-assisted process virtualization. In *Proceedings of the IEEE/IFIP Conference on Dependable Systems and Networks (DSN)*.
- [75] Koen Koning, Xi Chen, Herbert Bos, Cristiano Giuffrida, and Elias Athanasopoulos. 2017. No Need to Hide: Protecting Safe Regions on Commodity Hardware. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.
- [76] A. P. Kosoresow and S. A. Hofmeyer. 1997. Intrusion Detection via System Call Traces. *IEEE Software* (1997).
- [77] Per Larsen, Andrei Homescu, Stefan Brunthaler, and Michael Franz. 2014. SoK: Automated Software Diversity. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [78] Hojoon Lee, Chihyun Song, and Brent Byunghoon Kang. 2018. Lord of the x86 Rings: A Portable User Mode Privilege Separation Architecture on x86. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [79] Hojoon Lee, Chihyun Song, and Brent Byunghoon Kang. 2023. Harnessing the x86 Intermediate Rings for Intra-Process Isolation. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (2023), 3251–3268. doi:10.1109/TDSC.2022.3192524
- [80] Seongman Lee, Seoye Kim, Chihyun Song, Byeongsu Woo, Eunyeong Ahn, Junsu Lee, Yeongjin Jang, Jinsoo Jang, Hojoon Lee, and Brent Byunghoon Kang. 2024. GENESIS: A Generalizable, Efficient, and Secure Intra-kernel Privilege Separation. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*.
- [81] Hugo Lefeuvre, Nathan Dautenhahn, David Chisnall, and Pierre Olivier. 2025. SoK: Software Compartmentalization. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [82] Hugo Lefeuvre, Gauthier Gain, Vlad-Andrei Bădoiu, Daniel Dinca, Vlad-Radu Schiller, Costin Raiciu, Felipe Huici, and Pierre Olivier. 2024. Loupe: Driving the Development of OS Compatibility Layers.

- In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [83] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. 2018. Meltdown: Reading Kernel Memory from User Space. In *Proceedings of the USENIX Security Symposium*.
- [84] Yutao Liu, Tianyu Zhou, Kexin Chen, Haibo Chen, and Yubin Xia. 2015. Thwarting Memory Disclosure with Efficient Hypervisor-Enforced Intra-Domain Isolation. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [85] Kangjie Lu, Meng Xu, Chengyu Song, Taesoo Kim, and Wenke Lee. 2018. Stopping Memory Disclosures via Diversification and Replicated Execution. In *IEEE Transactions on Dependable and Secure Computing (TDSC)* (2018).
- [86] Chenke Luo, Jiang Ming, Mengfei Xie, Guojun Peng, and Jianming Fu. 2025. Retrofitting XoM for Stripped Binaries without Embedded Data Relocation. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.
- [87] Weiqin Ma, Pu Duan, Sanmin Liu, Guofei Gu, and Jyh-Charn Liu. 2012. Shadow Attacks: Automatically Evading System-Call-Behavior-Based Malware Detection. *Journal in Computer Virology* (2012).
- [88] Sanaya Malik, Narendra Singh, and Somanath Tripathy. 2026. Semantic characterization of android malware through runtime system call analysis. *Journal of Information Security and Applications* 98 (2026), 104406. doi:10.1016/j.jisa.2026.104406
- [89] Matthew Maurer and David Brumley. 2012. TACHYON: tandem execution for efficient live patch testing. In *Proceedings of the USENIX Security Symposium*.
- [90] Derrick Paul McKee, Yianni Giannaris, Carolina Ortega, Howard E Shrobe, Mathias Payer, Hamed Okhravi, and Nathan Burow. 2022. Preventing Kernel Hacks with HAKCs.. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.
- [91] Larry McVoy and Carl Staelin. 1996. Imbench: portable tools for performance analysis. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [92] A. Melvin, G. Jasper Kathrine, Andrew Jayabose, and Cennita David. 2025. A Deep Learning Model Leveraging Time-Series System Call Data to Detect Malware Attacks in Virtual Machines. *International Journal of Computational Intelligence Systems* (2025). doi:10.1007/s44196-025-00781-z
- [93] Akira Moroo, Hajime Tazaki, and Kenichi Yasukata. 2025. svc-hook: Hooking System Calls on ARM64 by Binary Rewriting. In *Proceedings of the ACM/IFIP International Middleware Conference (Middleware)*.
- [94] Pierre Olivier, Daniel Chiba, Stefan Lankes, Changwoo Min, and Binoy Ravindran. 2019. A binary-compatible unikernel. In *Proceedings of the ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE)*.
- [95] Sebastian Österlund, Koen Koning, Pierre Olivier, Antonio Barbalace, Herbert Bos, and Cristiano Giuffrida. 2019. kMVX: Detecting Kernel Information Leaks with Multi-variant Execution. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [96] Chengbin Pang, Ruotong Yu, Yaohui Chen, Eric Koskinen, Georgios Portokalidis, Bing Mao, and Jun Xu. 2021. SoK: All You Ever Wanted to Know About x86/x64 Binary Disassembly But Were Afraid to Ask. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [97] Abhinav Pathak, Y. Charlie Hu, Ming Zhang, Paramvir Bahl, and Yi-Min Wang. 2011. Fine-grained power modeling for smartphones using system call tracing. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.
- [98] Dinglan Peng, Congyu Liu, Tapti Palit, Rodrigo Fonseca, Anjo Vahldiek-Oberwagner, and Mona Vij. 2023. uSwitch: Fast Kernel Context Isolation with Implicit Context Switches. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [99] Dinglan Peng, Congyu Liu, Tapti Palit, Anjo Vahldiek-Oberwagner, Mona Vij, and Pedro Fonseca. 2025. Pegasus: Transparent and Unified Kernel-Bypass Networking for Fast Local and Remote Communication. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.
- [100] Luís Pina, Anastasios Andronidis, Michael Hicks, and Cristian Cadar. 2019. MVEDSU: Higher Availability Dynamic Software Updates via Multi-Version Execution. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [101] Luís Pina, Daniel Grumberg, Anastasios Andronidis, and Cristian Cadar. 2017. A DSL approach to reconcile equivalent divergent program executions. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [102] Sergej Proskurin, Marius Momeu, Seyedhamed Ghavamnia, Vasileios P. Kemerlis, and Michalis Polychronakis. 2020. xMP: Selective Memory Protection for Kernel and User Space. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [103] Niels Provos. 2003. Improving Host Security with System Call Policies. In *Proceedings of the USENIX Security Symposium*.
- [104] Vidya Lakshmi Rajagopalan, Konstantinos Kleftogiorgos, Enes Göktas, Jun Xu, and Georgios Portokalidis. 2023. SYSPART: Automated Temporal System Call Filtering for Binaries. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [105] Andreas Schmidt, Gregory Stock, Robin Ohs, Luis Gerhorst, Benedict Herzog, and Timo Hönig. 2024. carbond: An Operating-System Daemon for Carbon Awareness. *SIGENERGY Energy Inform. Rev.* 4, 3 (2024), 52–57.
- [106] David Schrammel, Samuel Weiser, Richard Sadek, and Stefan Mangard. 2022. Jenny: Securing Syscalls for PKU-Based Memory Isolation Systems. In *Proceedings of the USENIX Security Symposium*.
- [107] David Schrammel, Samuel Weiser, Stefan Steinegger, Martin Schwarzl, Michael Schwarz, Stefan Mangard, and Daniel Gruss. 2020. Donky: Domain Keys – Efficient In-Process Isolation for RISC-V and x86. In *Proceedings of the USENIX Security Symposium*.
- [108] Mark Seaborn and Thomas Dullien. 2015. Exploiting the DRAM rowhammer bug to gain kernel privileges. In *BlackHat USA*.
- [109] Yang Shen, Min Xie, Tao Wu, Wenzhe Zhang, Ruibo Wang, and Gen Zhang. 2025. ASC-Hook: Efficient System Call Interception for ARM. In *Proceedings of the 26th ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (Seoul, Republic of Korea) (LCTES '25)*. Association for Computing Machinery, New York, NY, USA, 73–84. doi:10.1145/3735452.3735524
- [110] Zhiming Shen, Zhen Sun, Gur-Eyal Sela, Eugene Bagdasaryan, Christina Delimitrou, Robbert Van Renesse, and Hakim Weatherspoon. 2019. X-Containers: Breaking Down Barriers to Improve Performance and Isolation of Cloud-Native Containers. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [111] Kevin Z. Snow, Fabian Monrose, Lucas Davi, Alexandra Dmitrienko, Christopher Liebchen, and Ahmad-Reza Sadeghi. 2013. Just-In-Time Code Reuse: On the Effectiveness of Fine-Grained Address Space Layout Randomization. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [112] Abel Souza, Noman Bashir, Jorge Murillo, Walid Hanafy, Qianlin Liang, David Irwin, and Prashant Shenoy. 2023. Ecovisor: A Virtual Energy System for Carbon-Efficient Applications. In *Proceedings of the International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.
- [113] Zahra Tarkhani and Anil Madhavapeddy. 2020. μ Tiles: Efficient Intra-Process Privilege Enforcement of Memory Regions. *arXiv preprint arXiv:2108.03705* (2020).

- [114] Martin Unterguggenberger, Lukas Lamster, David Schrammel, Martin Schwarzl, and Stefan Mangard. 2024. TME-Box: Scalable In-Process Isolation through Intel TME-MK Memory Encryption. *arXiv preprint arXiv:2407.10740v2*.
- [115] Anjo Vahldiek-Oberwagner, Eslam Elnikety, Nuno O. Duarte, Michael Sammler, Peter Druschel, and Deepak Garg. 2019. ERIM: Secure, Efficient In-Process Isolation with Protection Keys (MPK). In *Proceedings of the USENIX Security Symposium*.
- [116] Victor Van Der Veen, Yanick Fratantonio, Martina Lindorfer, Daniel Gruss, Clémentine Maurice, Giovanni Vigna, Herbert Bos, Kaveh Razavi, and Cristiano Giuffrida. 2016. Drammer: Deterministic rowhammer attacks on mobile platforms. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [117] Stijn Volckaert, Bart Coppens, and Bjorn De Sutter. 2016. Cloning your gadgets: Complete ROP attack immunity with multi-variant execution. In *IEEE Transactions on Dependable and Secure Computing (TDSC)* (2016).
- [118] Alexios Voulimeneas, Dokyung Song, Fabian Parzefall, Yeoul Na, Per Larsen, Michael Franz, and Stijn Volckaert. 2020. Distributed heterogeneous N-variant execution. In *In Proceedings of the Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*.
- [119] Alexios Voulimeneas, Jonas Vinck, Ruben Mechelinck, and Stijn Volckaert. 2022. You shall not (by) pass! practical, secure, and fast PKU-based sandboxing. In *Proceedings of the European Conference on Computer Systems (EuroSys)*.
- [120] Zhiyuan Wan, David Lo, Xin Xia, Liang Cai, and Shanping Li. 2017. Mining Sandboxes for Linux Containers. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*.
- [121] Qi Wang, Wajih Ul Hassan, Ding Li, Kangkook Jee, Xiao Yu, Kexuan Zou, Junghwan Rhee, Zhengzhang Chen, Wei Cheng, Carl A. Gunter, and Haifeng Chen. 2020. You Are What You Do: Hunting Stealthy Malware via Data Provenance Analysis. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.
- [122] Xiaoguang Wang, SengMing Yeoh, Robert Lyerly, Pierre Olivier, Sang-Hoon Kim, and Binoy Ravindran. 2020. A Framework for Software Diversification with ISA Heterogeneity. In *Proceedings of the International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*.
- [123] Xiaoguang Wang, SengMing Yeoh, Pierre Olivier, and Binoy Ravindran. 2020. Secure and Efficient In-Process Monitor (and Library) Protection with Intel MPK. In *Proceedings of the European Workshop on System Security (EuroSec)*.
- [124] Zhe Wang, Chenggang Wu, Mengyao Xie, Yinqian Zhang, Kangjie Lu, Xiaofeng Zhang, Yuanming Lai, Yan Kang, and Min Yang. 2020. SEIMI: Efficient and Secure SMAP-Enabled Intra-process Memory Isolation. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*.
- [125] Richard Wartell, Yan Zhou, Kevin W. Hamlen, Murat Kantarcioglu, and Bhavani Thuraisingham. 2011. Differentiating Code from Data in X86 Binaries. In *Proceedings of the 2011th European Conference on Machine Learning and Knowledge Discovery in Databases - Volume Part III*.
- [126] Chenggang Wu, Mengyao Xie, Zhe Wang, Yinqian Zhang, Kangjie Lu, Xiaofeng Zhang, Yuanming Lai, Yan Kang, Min Yang, and Tao Li. 2023. Dancing With Wolves: An Intra-Process Isolation Technique With Privileged Hardware. *IEEE Trans. Dependable Secur. Comput.* 20, 3 (May 2023), 1959–1978. doi:10.1109/TDSC.2022.3168089
- [127] Mengyao Xie, Chenggang Wu, Yinqian Zhang, Jiali Xu, Yuanming Lai, Yan Kang, Wei Wang, and Zhe Wang. 2022. CETIS: Retrofitting Intel CET for Generic and Efficient Intra-process Memory Isolation. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [128] Jiali Xu, Mengyao Xie, Chenggang Wu, Yinqian Zhang, Qijing Li, Xuan Huang, Yuanming Lai, Yan Kang, Wei Wang, Qiang Wei, and Zhe Wang. 2023. PANIC: PAN-assisted Intra-process Memory Isolation on ARM. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*.
- [129] Meng Xu, Kangjie Lu, Taesoo Kim, and Wenke Lee. 2017. Bunshin: Compositing Security Mechanisms through Diversification. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [130] Fangfei Yang, Bumjin Im, Weijie Huang, Kelly Kaoudis, Anjo Vahldiek-Oberwagner, Chia-Che Tsai, and Nathan Dautenhahn. 2024. Endokernel: A Thread-Safe Monitor for Lightweight Subprocess Isolation. In *Proceedings of the USENIX Security Symposium*.
- [131] Kenichi Yasukata, Hajime Tazaki, Pierre-Louis Aublin, and Kenta Ishiguro. 2023. zpoline: A System Call Hook Mechanism Based on Binary Rewriting. In *Proceedings of the USENIX Annual Technical Conference (ATC)*.
- [132] Hengkai Ye, Song Liu, Zhechang Zhang, and Hong Hu. 2023. VIPER: Spotting Syscall-Guard Variables for Data-Only Attacks. In *Proceedings of the USENIX Security Symposium*.
- [133] Sengming Yeoh, Xiaoguang Wang, Jae-Won Jang, and Binoy Ravindran. 2024. sMVX: Multi-Variant Execution on Selected Code Paths. In *Proceedings of the ACM/IFIP International Middleware Conference (Middleware)*.
- [134] Irene Zhang, Amanda Raybuck, Pratyush Patel, Kirk Olynnyk, Jacob Nelson, Omar S. Navarro Leija, Ashlie Martinez, Jing Liu, Anna Kornfeld Simpson, Sujay Jayakar, Pedro Henrique Penna, Max Demoulin, Piali Choudhury, and Anirudh Badam. 2021. The Demikernel Datapath OS Architecture for Microsecond-Scale Datacenter Systems. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*.
- [135] Mingwei Zhang, Ravi Sahita, and Daiping Liu. 2018. eXecutable-Only-Memory-Switch (XOM-Switch). In *Black Hat Asia Briefings (Black Hat Asia)*.
- [136] Qihang Zhou, Wenzhuo Cao, Xiaoqi Jia, Peng Liu, Shengzhi Zhang, Jiayun Chen, Shaowen Xu, and Zhenyu Song. 2025. RContainer: A Secure Container Architecture through Extending ARM CCA Hardware Primitives. In *Proceedings of the Symposium on Network and Distributed System Security (NDSS)*.

A Fuzzing Heuristics for Syscall Instruction Discovery

Our fuzzing approach (see Section 5.1) is designed to enumerate syscall instructions invoked by a program rather than to identify crashes or software bugs. We instrument the application binary to provide AFL++ with edge-coverage feedback. Rather than tracking coverage of the libraries themselves, we use the application’s execution paths as a proxy: inputs that drive the application into previously unexplored branches are more likely to exercise new call patterns into `libc.so` and `ld.so`, thereby uncovering syscall invocation sites that would otherwise remain unvisited. Shared libraries such as `libc.so` and `ld.so` remain uninstrumented, as they are the primary source of system calls in practice and their offsets are stable and directly portable to production binaries. As reported in Section 7, we set one hour as the hard time limit for all campaigns; all campaigns in our evaluation saturated well before this limit, with no new syscall instructions discovered after convergence. Across all campaigns, our approach identified 144 unique syscall instructions across `libc.so` and

ld. so. Across all microbenchmark and macrobenchmark experiments (see Section 7), we did not observe a single system call falling back to the SUD-based slow path, confirming that our fuzzing approach achieved sufficient syscall coverage for these workloads.

Mechanism. AFL++ executes test cases using a forkserver model: a template process is initialized once and individual test cases are run by forking from it, avoiding the overhead of re-invoking `exec()` for every input. For each forked child, SUD is enabled to intercept executing syscall instructions. When a syscall instruction executes, SUD delivers a SIGSYS signal to a handler that records the instruction’s runtime address. This address is then resolved to a (library path, offset) pair by consulting `/proc/self/maps`, yielding offsets that are stable across runs and directly usable for binary rewriting of production binaries. Discovered offsets are communicated back to the fuzzer via shared memory and logged for later use. We describe the two heuristics below.

Edge-Coverage Heuristic. We leverage a concept analogous to AFL++’s edge coverage, but instead of tracking new branch transitions, we treat each newly discovered syscall instruction offset as a coverage edge. After each execution, our fuzzer examines newly observed offsets and determines whether the current input is a better representative for any

of them, preferring inputs that are smaller or execute faster. Our fuzzer maintains such a representative seed for each discovered offset and assigns additional energy to seeds that uncover new offsets, increasing their likelihood of being selected for further mutation. This biases exploration toward inputs that exercise previously unseen syscall invocation sites, making it particularly effective for short fuzzing campaigns where each execution must count.

Region Heuristic. Whenever a new syscall instruction is discovered in a non-instrumented library, we create a region spanning $\pm N$ bytes around its offset (default: $N = 1024$). We use a disassembler (`libopcodes`) to statically estimate the number of syscall instructions within that region and assign the region an initial energy value proportional to this estimate. Since this estimate guides only initial energy allocation rather than actual syscall discovery — which is confirmed at runtime through SUD — the inherent imprecision of static disassembly on x86-64 does not affect correctness. This energy is gradually reduced as syscall instructions within the region are confirmed during fuzzing. As a result, the fuzzer prioritizes unexplored library regions over time, shifting focus away from areas where all syscall sites have already been identified, and toward regions where unconfirmed sites likely remain.